МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский авиационный институт (национальный исследовательский университет)»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
№ 4

Дисциплина:
Операционные системы и архитектура компьютера

Тема:
Отображаемые файлы и взаимодействие процессов. Вариант 6

Выполнил:
студент группы М8О-206БВ-25
Крылов Д.Ф.

Проверил:
Ассистент Центра ТОП-ИТ
Ефимов Д.А.

Москва, 2026 г.

1. Цель работы

Цель лабораторной работы - реализовать взаимодействие двух процессов в Windows с помощью отображаемой памяти и событий синхронизации. Программа должна решать задачу варианта 6 из лабораторной работы № 2: читать из файла строки с целыми числами произвольного количества, вычислять сумму чисел каждой строки и выводить результаты в консоль родительского процесса.

В ходе работы необходимо создать дочерний процесс, передать ему входной файл как стандартный поток ввода, организовать общую область памяти, согласовать запись и чтение результатов и обработать ошибки системных вызовов. Итогом служит программа на языке C, состоящая из отдельных исполняемых файлов parent.exe и child.exe. Для каждой непустой строки входного файла родитель выводит одну сумму.

2. Краткое описание архитектуры решения

Решение состоит из родительской программы parent.exe, дочерней программы child.exe и общего заголовка shared.h. Родитель принимает путь к входному файлу, открывает его через CreateFileA и создаёт дочерний процесс вызовом CreateProcessA. В структуре STARTUPINFOA открытый файл назначается стандартным вводом child.exe, следовательно, дочерняя программа читает файл обычным fgetc(stdin), не открывая его повторно. Родитель остаётся ответственным за вывод сумм в консоль [2].

Вместо канала, применявшегося во второй лабораторной работе, используется именованный объект отображения памяти. Родитель создаёт его функцией CreateFileMappingA и отображает в своё адресное пространство через MapViewOfFile. Дочерний процесс открывает тот же объект по имени через OpenFileMappingA и также получает указатель на его содержимое. Объект создан с дескриптором INVALID_HANDLE_VALUE: общая область поддерживается файлом подкачки Windows и не создаёт отдельный файл результатов на диске [1].

2.1 Схема взаимодействия компонентов

Входной файл содержит команды вида «число число ... конец строки». После запуска ребёнок последовательно считывает символы, выделяет целые числа, накапливает сумму и передаёт готовый результат родителю. Передача каждой суммы использует один и тот же участок общей памяти; завершение файла и ошибка представлены отдельными типами сообщений.

```
input.txt -> stdin(child.exe)
child.exe -> SharedData -> ready -> parent.exe -> stdout
child.exe <- received <- parent.exe
```

Рисунок 1 - Поток данных и событий между процессами

2.2 Формат общей памяти и синхронизация

Структура SharedData содержит поле kind с типом сообщения, поле sum со значением суммы и поле line с номером обрабатываемой строки. Допустимы сообщения MESSAGE_RESULT, MESSAGE_DONE и MESSAGE_ERROR. Единый заголовочный файл подключён к обеим программам, поэтому расположение полей одинаково для родителя и ребёнка. Номер строки передаётся для указания места ошибки.

Два именованных события образуют подтверждение передачи. После записи структуры ребёнок подаёт сигнал ready и ожидает received. Родитель ждёт ready, читает структуру, печатает сумму либо обрабатывает завершение или ошибку, затем подаёт сигнал received. Только после этого ребёнок может записать следующий результат в тот же участок памяти. Оба события создаются в автоматическом режиме сброса. Родитель дополнительно ожидает дескриптор дочернего процесса: если тот завершится до итогового сообщения, ожидание события не останется бесконечным [3].

2.3 Обработка входных данных и ошибок

Дочерний процесс формирует токен из непробельных символов и преобразует его функцией strtol. Проверяется, что токен целиком является числом и что значение помещается в диапазон int. Сумма хранится в long long для безопасного вычисления промежуточного результата; после каждого прибавления проверяется диапазон int. Перевод строки отделяет одну команду от следующей. Пустые строки пропускаются, а последняя строка обрабатывается и без завершающего перевода строки.

При ошибке открытия файла родитель сообщает код GetLastError и не запускает ребёнка. При неверном числе, переполнении суммы или ошибке чтения ребёнок пишет причину в stderr, посылает MESSAGE_ERROR и завершает работу с кодом 1. Родитель подтверждает получение сообщения, дожидается ребёнка, проверяет его код завершения и тоже возвращает 1. Перед выходом закрываются дескрипторы файла, отображения, событий, процесса и потока; отображённые области снимаются через UnmapViewOfFile.

3. Листинг ключевых фрагментов кода

Ниже приведены фрагменты, которые показывают создание процесса, организацию общей памяти, передачу результата и проверку входных чисел. Полные исходные файлы src/parent.c, src/child.c, src/shared.h находятся в каталоге проекта. Адрес GitHub-репозитория: https://github.com/SpectralBlade/Lab_OS_4.

3.1 Передача файла дочернему процессу

Родитель назначает дескриптор открытого файла стандартным вводом ребёнка. Параметр TRUE в CreateProcessA разрешает наследование этого дескриптора. Имена отображения и событий передаются через командную строку.

```
startup.cb = sizeof(startup);
startup.dwFlags = STARTF_USESTDHANDLES;
startup.hStdInput = file;
startup.hStdOutput = GetStdHandle(STD_OUTPUT_HANDLE);
startup.hStdError = GetStdHandle(STD_ERROR_HANDLE);
if (!CreateProcessA(child_path, command, NULL, NULL, TRUE, 0,
                    NULL, NULL, &startup, &child)) {
    fprintf(stderr, "CreateProcess failed: %lu\n", GetLastError());
    goto cleanup;
}
```

Листинг 1 - Настройка стандартного ввода и запуск дочернего процесса

3.2 Создание отображения памяти

Родитель создаёт отображение размером с одно сообщение SharedData и открывает его представление для чтения и записи. Проверка ERROR_ALREADY_EXISTS не позволяет случайно использовать объект, оставшийся под тем же именем.

```
mapping = CreateFileMappingA(INVALID_HANDLE_VALUE, NULL,
                             PAGE_READWRITE, 0,
                             sizeof(SharedData), map_name);
if (mapping == NULL || GetLastError() == ERROR_ALREADY_EXISTS) {
    fprintf(stderr, "CreateFileMapping failed or name exists: %lu\n",
            GetLastError());
    goto cleanup;
}
data = MapViewOfFile(mapping, FILE_MAP_ALL_ACCESS,
                     0, 0, sizeof(*data));
```

Листинг 2 - Создание именованной общей области памяти

3.3 Передача одной суммы

Функция send_message записывает сообщение, «будит» родителя и ждёт подтверждения. Таким образом, один участок памяти используется многократно без потери результата.

```
data->kind = kind;
data->sum = sum;
data->line = line;
if (!SetEvent(ready)) {
    fprintf(stderr, "SetEvent(ready) failed: %lu\n", GetLastError());
    return 0;
}
if (WaitForSingleObject(received, INFINITE) != WAIT_OBJECT_0) {
    fprintf(stderr, "WaitForSingleObject(received) failed: %lu\n",
            GetLastError());
    return 0;
}
```

Листинг 3 - Запись сообщения и ожидание подтверждения

3.4 Проверка числа и промежуточной суммы

Чтение файла не ограничивает количество чисел в строке. Каждое число проверяется перед прибавлением; некорректный токен или выход суммы за диапазон int прекращает обработку с сообщением об ошибке.

```
errno = 0;
number = strtol(token, &end, 10);
if (end == token || *end != '\0' || errno == ERANGE ||
    number < INT_MIN || number > INT_MAX) {
    return 0;
}
*sum += number;
return *sum >= INT_MIN && *sum <= INT_MAX;
```

Листинг 4 - Преобразование числа и контроль диапазона

4. Результаты работы

4.1 Исходные данные и результат

Для основного запуска использован файл src/input.txt, скопированный CMake в cmake-build-debug/input.txt. Первая строка содержит 1, 2 и 3; вторая - 10 и -4; третья - 7. Суммы равны соответственно 1 + 2 + 3 = 6, 10 + (-4) = 6 и 7. После ввода имени файла программа вывела три строки:

```
6
6
7
```

Листинг 5 - Результат обработки файла input.txt

Число строк вывода совпало с числом непустых строк входного файла. Завершение обеих программ прошло с кодом 0. Суммы выводятся родительским процессом после получения сообщений MESSAGE_RESULT из общей памяти.

4.2 Проверка особых случаев и ошибок

Помимо основного примера выполнены запуски с последней строкой без символа перевода, длинной строкой из 5000 единиц, строкой ровно из 5000 символов, пустым файлом, неверным токеном, слишком длинным числом, переполнением суммы и отсутствующим файлом. Таблица 1 фиксирует фактически полученные результаты и коды завершения parent.exe.

Таблица 1 - Результаты проверочных запусков

Входной случай	**Результат**	**Код**
input.txt: 3 строки	6; 6; 7	0
Последняя строка «2 3» без перевода	5	0
5000 чисел, каждое равно 1	5000	0
Строка из 5000 символов: 2500 единиц	2500	0

Входной случай	Результат	Код
Пустой файл	Вывод отсутствует	0
Строка «1 abc»	Ошибка числа, строка 1	1
Число из 64 цифр	Слишком длинное число, строка 1	1
Строка «2147483647 1»	Переполнение суммы, строка 1	1
Отсутствующий файл	Ошибка CreateFile, код Windows 2	1

Проверочные файлы находятся в каталоге tests: last_line_no_newline.txt, 5000_ones.txt, long_line_5000_chars.txt, empty.txt, bad_token.txt, too_long_number.txt и overflow.txt. CMake копирует их в cmake-build-debug/tests. Для проверки ошибки открытия вводится tests/missing.txt: этот файл намеренно не создан.

На неверном и слишком длинном токене, а также при переполнении ребёнок завершил вычисление на первой строке, передал MESSAGE_ERROR и вернул код 1. Если файл не существует, ошибка возникает ещё в родителе при CreateFileA, поэтому ребёнок не запускается. Проверка с 5000 числами подтверждает, что количество чисел в одной строке не привязано к размеру буфера токена.

4.3 Скриншоты результатов

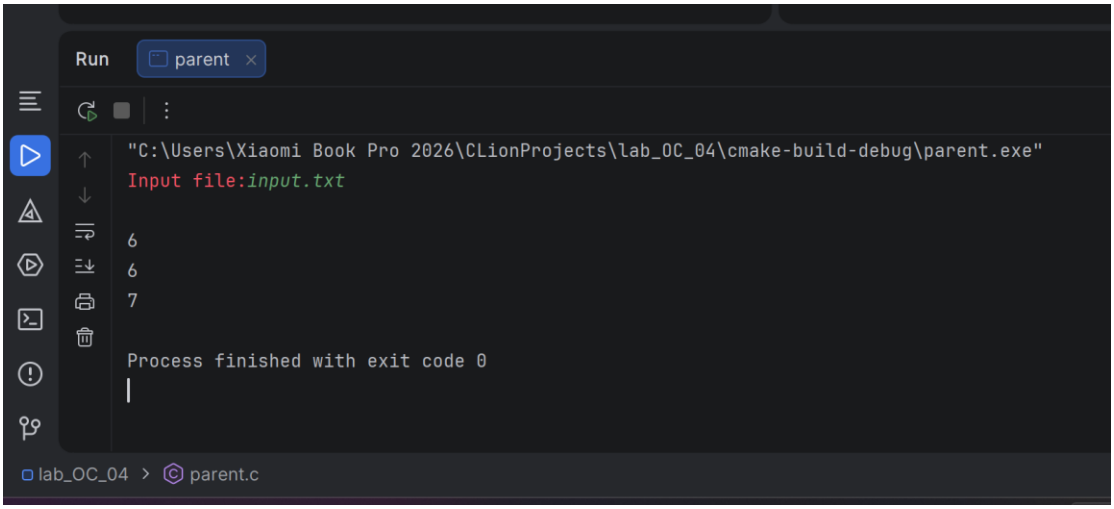


Рисунок 2 - Успешный запуск программы для файла input.txt

Для следующих снимков в CLion запускается цель parent с рабочим каталогом cmake-build-debug. На запрос Input file: вводится относительное имя файла, указанное под каждым снимком.

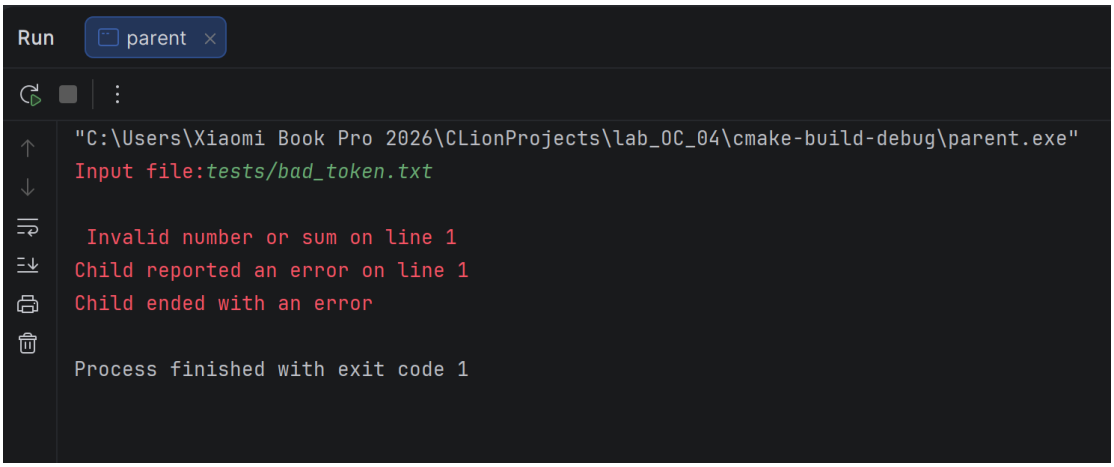


Рисунок 3 - Ошибка преобразования числа

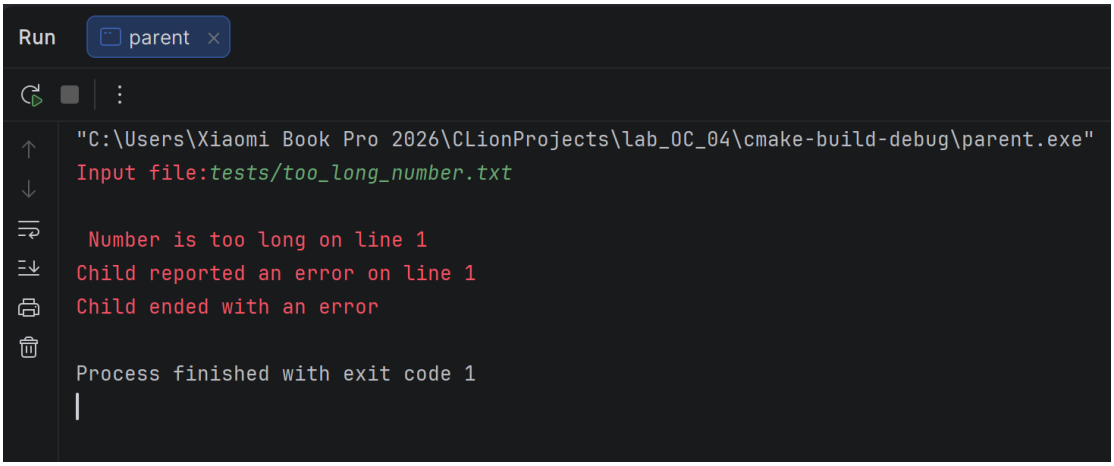


Рисунок 4 - Ошибка слишком длинного числа



Рисунок 5 - Ошибка открытия отсутствующего файла

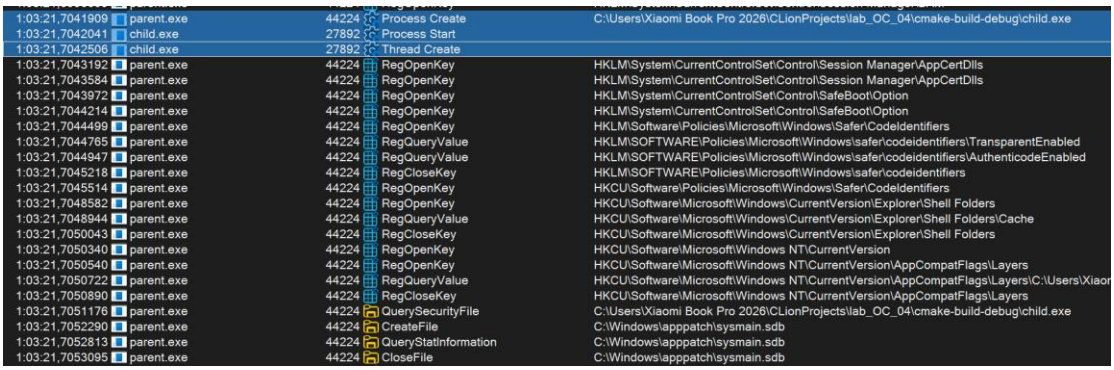


Рисунок 6 - Родительский и дочерний процессы в Process Monitor

4.4 Анализ трассы Process Monitor

В файле extras/Logfile.CSV сохранена трасса запуска. В 1:03:21,6894061 parent.exe (PID 44224) открыл cmake-build-debug/input.txt: операция CreateFile завершилась с результатом SUCCESS. В 1:03:21,7041909 родитель создал child.exe (PID 27892). Следующая запись Process Start дочернего процесса содержит Parent PID 44224.

В поле Detail события Process Create записаны аргументы child.exe: Local\LabOC4Map_44224, Local\LabOC4Ready_44224 и Local\LabOC4Received_44224. Эти имена формируются в src/parent.c для отображаемой памяти и двух событий синхронизации.



Рисунок 7 – Аргументы при запуске дочернего процесса child.exe

В 1:03:21,7237246 child.exe прочитал из input.txt 14 байт (ReadFile, SUCCESS); следующее чтение вернуло END OF FILE. В 1:03:21,7252443 завершился child.exe, а в 1:03:21,7268462 — parent.exe. Для обоих процессов в трассе указан Exit Status 0. Размер src/input.txt также составляет 14 байт. Трасса показывает последовательность создания дочернего процесса, чтения входного файла и штатного завершения обеих программ.

5. Выводы

В лабораторной работе реализованы две программы на языке C для Windows. Родитель открывает входной файл и создаёт дочерний процесс с перенаправленным стандартным вводом. Ребёнок вычисляет сумму целых чисел для каждой непустой строки и передаёт результат через именованное отображение памяти. Два события согласуют запись и чтение каждого сообщения.

Для файла из трёх строк получены суммы 6, 6 и 7. Дополнительные проверки показали обработку файла без завершающего перевода строки, большого числа элементов в строке, пустого файла, неверного токена, переполнения и отсутствия входного файла. При штатном завершении ресурсы освобождаются, а при ошибках программы возвращают ненулевой код. Работа закрепила применение CreateProcessA, CreateFileMappingA, MapViewOfFile, именованных событий и ожидания процесса.

6. Использованные источники

[1] Microsoft Learn. Sharing Files and Memory.
[2] Microsoft Learn. Creating a Child Process with Redirected Input and Output.
[3] Microsoft Learn. Using Named Objects.